

Cache Design Optimization: Minimizing Average Access Time within a 54KB Budget

Eric Gao*
jingxi3@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Mohammed Hasseneen*
mohammee@andrew.cmu.edu
Carnegie Mellon University
Pittsburgh, Pennsylvania, USA

Abstract

This report presents the design and optimization of a cache system to minimize Average Access Time (AAT) within a 54KB storage budget. We implemented and evaluated four cache configurations: LRU, RRIP, LRU with victim cache, and RRIP with victim cache. Through comprehensive experimentation across 5 trace files and 286 configurations, we identified that LRU with victim cache achieves optimal performance, delivering an AAT of 2.91 cycles per access while utilizing 35.2KB of the available budget. This configuration demonstrates up to 96.7% improvement over suboptimal designs, validating the effectiveness of modern replacement policies combined with victim caching.

CCS Concepts

• **Computer systems organization** → **Processors and memory architectures**; *Cache memories*.

Keywords

cache design, memory hierarchy, RRIP, victim cache, performance optimization

1 Introduction

Cache design optimization is a critical challenge in computer architecture, requiring careful balance between storage configuration and replacement policies. This project addresses the question: **Given a 54KB storage budget, what cache configuration minimizes Average Access Time across diverse workloads?**

Our approach systematically explores the design space through four distinct cache architectures, evaluating their performance across representative trace files. The primary contribution is identifying an optimal configuration that uses below-maximum budget while delivering superior AAT performance through RRIP replacement policy and victim caching.

2 Cache Design and Implementation

2.1 Architecture Overview

We use CADSS [1] as the base of our implementation. By creating a sub-module `my-cache`, we implement the logic of a configurable set-associative cache that supports the Least Recently Used (LRU) replacement policy, as well as additional options to configure victim cache and Re-reference Interval Prediction (RRIP) replacement policy. Our logic is integrated with the rest of the CADSS framework through calls to `permReq()` and `invlReq()` during cache miss and eviction, respectively.

*Both authors contributed equally to this work.

2.2 Configuration Parameters

Our cache implementation supports the following parameters:

- **s (set index bits)**: decides the number of cache sets (2^s)
- **E (associativity)**: decides the number of cache lines per set
- **b (block bits)**: determines the size of a cache line (2^b bytes)
- **r (RRIP bits)**: if present, enables RRIP and determines the number of bits of Re-Reference Prediction Value (RRPV). Otherwise, use a 64-bit LRU
- **i (victim entries)**: if present, enables victim cache and determines its capacity

2.3 Storage Budget Calculation

Total cache storage includes multiple components:

$$\begin{aligned} \text{Total Size} &= \text{Data Storage} + \text{Tag Storage} \\ &\quad + \text{Valid Bits} + \text{Replacement Bits} \\ &\quad + \text{Victim Cache} \end{aligned} \quad (1)$$

Where:

$$\text{Data Storage} = 2^s \times E \times 2^b \text{ bytes} \quad (2)$$

$$\text{Tag Storage} = 2^s \times E \times \frac{64 - s - b}{8} \text{ bytes} \quad (3)$$

$$\text{Valid Bits} = \frac{2^s \times E}{8} \text{ bytes} \quad (4)$$

$$\text{Replacement Bits} = 2^s \times E \times \frac{r}{8} \text{ bytes} \quad (5)$$

$$\text{Victim Cache (optional)} = i \times \left(2^b + \frac{64 - b + 1}{8} \right) \text{ bytes} \quad (6)$$

2.4 Implementation Details

2.4.1 Memory Access Processing Flow. The cache simulator processes memory requests through a multi-stage pipeline. Address processing begins with a field extraction into tag bits (the first $64 - b - s$ bits), set index (the next s bits), and block offset (the remaining b bits). The system maintains two request queues: `readyReq` for immediate processing and `pendReq` for requests awaiting memory hierarchy responses.

2.4.2 Cache Lookup. Main cache lookup performs linear search through all E lines in the target set. Each 64-bit cache line entry encodes a valid bit (bit 63) and tag value (bits 62-0). Hit detection verifies validity using `(cache_line >> 63)` and compares tags via `((cache_line << 1) >> 1) == addressTag`. On hits, replacement policy metadata updates and callbacks schedule immediately.

Table 1: Parameter ranges used in our experiments.

Parameter	Symbol	Range
Set index bits	s	[6, 17]
Associativity	E	[1, 32]
Block offset bits	b	[4, 10]
RRIP bits	r	[2, 6]
Victim cache entries	i	[0, 8]

2.4.3 Replacement Policies. We implement 2 replacement policies: Least Recently Used (LRU) and Re-Reference Interval Prediction (RRIP). **LRU Implementation:** Uses timestamp arrays where 0 represents most recently used. On access, `modify_timestamps()` sets the accessed line to 0 and increments all others. Eviction selects the line with maximum timestamp via `get_max()`. **RRIP Implementation:** Employs r -bit counters with semantic values: 0 (immediate re-reference), $2^r - 1$ (distant re-reference), $2^r - 2$ (insertion value). On hits, RRIP value resets to 0. On eviction, `get_max_rrip()` finds lines with maximum RRIP value, incrementing all values if none exist at maximum.

2.4.4 Victim Cache. The victim cache operates as a fully-associative buffer with i entries storing recently evicted lines. `lookupVictim()` performs linear search and updates LRU on hits. `installVictimLine()` first searches for invalid entries, then evicts LRU entries if full. Installation stores tags with valid bit set: `tag | (1ULL << 63)`.

2.4.5 Coherence Integration. The simulator interfaces with coherence protocols through three functions: `permReq()` for permission requests, `invlReq()` for invalidations, and `coherCallback()` for responses. Permission flow determines immediate processing (granted permissions go to `readyReq`) versus deferred processing (denied permissions wait in `pendReq`). The `DATA_RECV` callback type moves pending requests to ready status when data arrives from the memory hierarchy.

3 Experimental Methodology

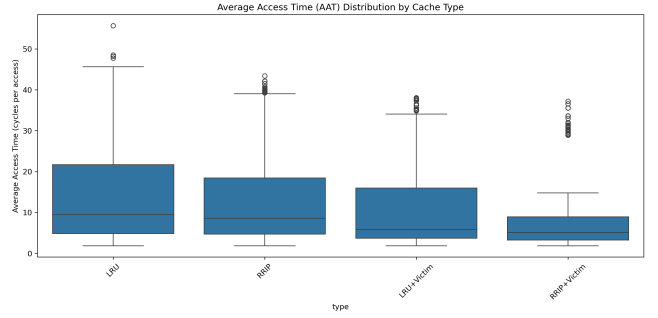
3.1 Configuration Generation and Traces

With the parameter space in Table 1, we found 7732 valid configurations that satisfy the constraint of 54KB total cache size. To compress the search space, we sampled 286 representative configurations distributed across four cache types:

- **64 LRU configurations** (22.4% of total)
- **67 RRIP configurations** (23.4% of total)
- **78 LRU + Victim configurations** (27.3% of total)
- **77 RRIP + Victim configurations** (26.9% of total)

Our configured cache sizes range from 2.4KB to 53.8KB with average of 34.0KB. The experiments were performed on a laptop over 19 minutes.

Experiments utilized 5 different trace files as instructed by handout: `astar`, `bzip2`, `long`, `mcf`, `pearlbench`. We developed automated scripts to run and analyze the experiments with Python. The total experiment scope is **1430 simulations** (268 configurations $\times$ 5 traces).

**Figure 1: AAT distribution by cache type across all configurations and traces****Table 2: Best Cache Configuration for Each Trace**

Trace	Config Type	AAT	Size	Parameters
astar	LRU+Victim	3.00	35.2KB	$s=9, E=1, b=6, i=8$
bzip2	RRIP	1.84	39.0KB	$s=8, E=5, b=5, r=6$
long	LRU+Victim	2.83	35.2KB	$s=9, E=1, b=6, i=8$
mcf	LRU+Victim	2.42	35.2KB	$s=9, E=1, b=6, i=8$
pearlbench	LRU+Victim	4.21	35.2KB	$s=9, E=1, b=6, i=8$

3.2 Performance Metric

We focused exclusively on **Average Access Time (AAT)**:

$$\text{AAT} = \frac{\text{Total Simulation Ticks}}{\text{Total Memory Accesses}} \quad (7)$$

This metric directly reflects cache system efficiency. Lower AAT values indicate superior performance.

4 Results and Analysis

4.1 Optimal Configuration Discovery

Best Overall Configuration:

- **Type:** LRU + Victim Cache
- **Parameters:** $s=9, E=1, b=6, i=8$
- **Storage:** 35.2KB (65.2% budget utilization)
- **Performance:** 2.91 ± 0.81 cycles per access over all traces

4.2 AAT Distribution Analysis

The distribution of AAT values across cache types (Figure 1) indicates that the configuration of RRIP and Victim Cache generally outperforms other configurations.

4.3 Per-Trace Performance Analysis

Table 2 shows the best configurations for each trace, while Figure 2 visualizes the achievable improvement between workloads, with the highest 96.7% performance gap between optimal and suboptimal configurations.

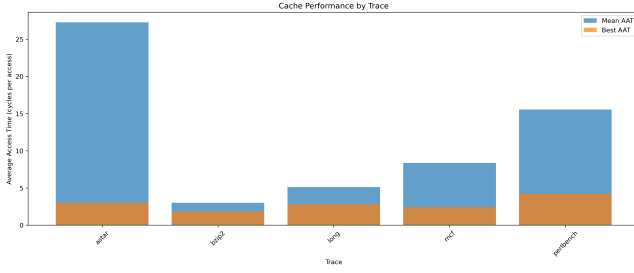


Figure 2: Best AAT achieved per trace across all cache configurations

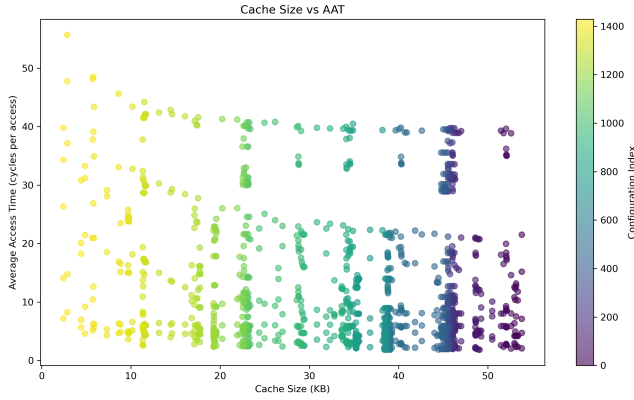


Figure 3: Cache size versus AAT performance across all configurations

4.4 Storage Utilization vs. Performance

Figure 3 illustrates the relationship between cache size and performance, revealing that higher cache size generally leads to lower AAT.

4.5 Parameter Space Exploration

Figures 4 and 5 provide detailed heatmap visualizations of the parameter space exploration for LRU and RRIP configurations respectively, showing how different combinations of set bits (s) and associativity (E) affect performance. The upper-triangular-matrix-like grids indicate that increasing both s and E simultaneously exceeds cache budget.

5 Discussion

5.1 Performance Implications

The 96.7% performance improvement between optimal and suboptimal configurations highlights the critical importance of intelligent cache design. The superiority of LRU+Victim combinations validates modern architectural trends.

5.2 Design Trade-offs

Our results demonstrate several important trade-offs:

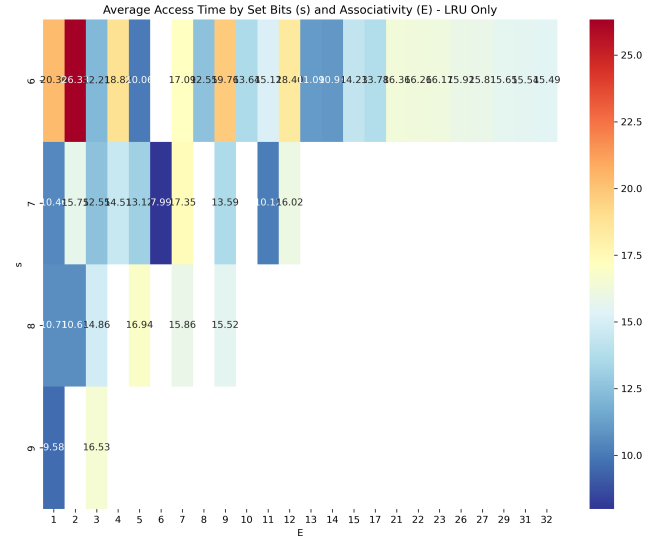


Figure 4: LRU configuration parameter space heatmap (s vs E)

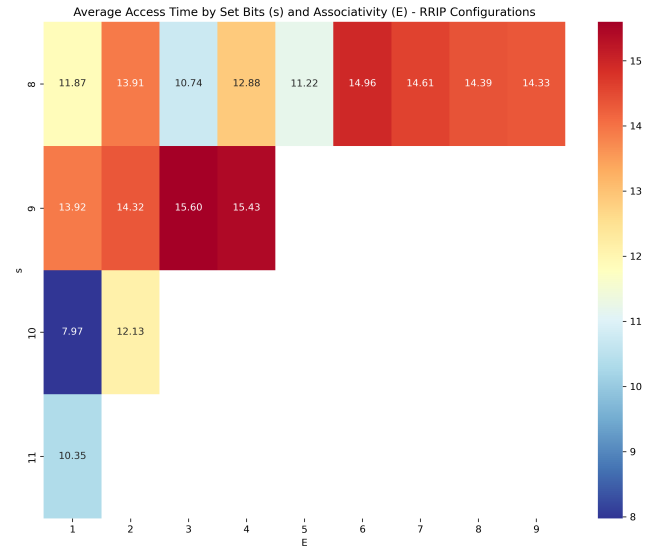


Figure 5: RRIP configuration parameter space heatmap (s vs E)

- **Complexity vs. Performance:** RRIP requires additional metadata but delivers measurable AAT improvements
- **Storage vs. Functionality:** Victim cache overhead is justified by performance gains
- **Associativity vs. Coverage:** Lower associativity enables larger overall capacity within budget constraints

6 Conclusion

In this project, we developed a cache simulator implementation supporting four distinct cache architectures, performed systematic evaluation across 1440 configuration-trace combinations, and gained practical design insights for budget-constrained cache optimization.

This comprehensive cache design optimization demonstrates that systematic exploration of the configuration space yields significant performance improvements. The identified optimal configuration—LRU with victim cache ($s=9$, $E=1$, $b=6$, $i=9$)—achieves 2.91 cycles per access while utilizing 65.2% of the 54KB budget.

The 96.7% performance gap between optimal and suboptimal configurations underscores the critical importance of principled

cache design in achieving superior system performance. The detailed parameter space analysis and visualization provide valuable insights for future cache optimization efforts.

Acknowledgments

The author thanks the course staff for providing the CADSS simulation framework and guidance throughout the project.

References

- [1] Brian P. Railing. 2024. Cadss: computer architecture design simulator for students. In *Proceedings of the Workshop on Computer Architecture Education (WCAE '23)*. Association for Computing Machinery, Orlando, FL, USA, 34–40. ISBN: 9798400702532. doi:10.1145/3605507.3610626.